

Proyecto Simulación de un Sistema de Reservas - Sistemas Operativos



**Juliana Aguirre Ballesteros
Juan Carlos Santamaria Orjuela
Juan David Daza**

**Pontificia Universidad Javeriana
Facultad de ingeniería
Departamento de ingeniería de sistemas
Bogotá D.C.
2025**

ÍNDICE

1. Introducción	2
1.1 Contexto del problema.....	3
1.2 Objetivos del proyecto	3
1.3 Alcance y limitaciones	4
2.Descripción del problema	5
2.1 Descripción general del sistema de reservas	5
2.2 Controlador de Reserva (servidor)	5
2.3 Agentes de Reserva (clientes)	6
2.4 Reglas de negocio y condiciones del sistema	10
2.5 Supuestos y restricciones	12
3.Diseño del sistema	12
3.1 Arquitectura general	13
3.2 Diseño del servidor	14
3.3 Diseño de los agentes	16
3.4 Comunicación entre procesos.....	17
3.5 Sincronización y concurrencia	19
3.6 Manejo de errores y validaciones	21
4.Implementación.....	22
4.1 Lenguaje, compilador y entorno de desarrollo.....	23
4.2 Estructura de archivos del proyecto	23
4.3 Instrucciones de compilación.....	25
4.4 Instrucciones de ejecución del servidor.....	26

4.5 Instrucciones de ejecución de los agentes	26
5. Plan de pruebas	27
5.1 Objetivo de las pruebas.....	27
5.2 Entorno de pruebas	28
5.3 Diseño de casos de prueba	28
5.4 Resultados de las pruebas	28
5.4.1 Compilación	28
5.4.2 Servidor - Validación de argumentos	29
5.4.2.1 Caso 1	30
5.4.2.2 Caso 2	30
5.4.2.3 Caso 3	30
5.4.2.4 Caso 4	30
5.4.2.5 Caso 5	30
5.4.3 Agente – Validación de argumentos.....	31
5.4.3.1 Caso 1	31
5.4.3.2 Caso 2	31
5.4.3.3 Caso 3	31
5.4.4 Handshake Agente-Servidor	31
5.4.5 Múltiples agentes concurrentes.....	32
5.4.6 Reservas aceptadas/reprogramadas/negadas	33
5.4.7 Reloj de simulación	34
5.4.8 Entradas/salidas por hora	34
5.4.9 Fin del día	34
5.5 Análisis de resultados	35

6.Conclusiones.....	35
7. Archivos de entrada de ejemplo	36

1. Introducción

Propósito y alcance

Este documento ofrece una introducción completa al sistema de reservas del parque, una aplicación cliente-servidor diseñada para simular la gestión de aforo en función del tiempo en una instalación recreativa. El sistema coordina múltiples procesos de cliente concurrentes agente que envían solicitudes de reserva a un servidor central servidor, el cual aplica las restricciones de aforo durante las horas de funcionamiento simuladas.

Esta página abarca la arquitectura fundamental, los componentes principales, los mecanismos de comunicación y el modelo de ejecución. Para obtener información detallada sobre subsistemas específicos, consulte:

Construcción e implementación del sistema: Primeros pasos

Patrones arquitectónicos detallados: Arquitectura del sistema

Detalles de implementación de IPC: Protocolo de comunicación

Funcionamiento interno del servidor: Proceso del servidor

Implementación del cliente: Proceso del agente

1.1 Contexto del problema

Durante la temporada vacacional, el parque privado Berlín recibe una gran cantidad de familias en un espacio reducido, lo que provoca congestión en los servicios y dificultades para controlar el aforo máximo permitido. Para mejorar esta situación, se plantea la necesidad de un sistema de reservas por horas que permita a la administración conocer cuántas personas ingresarán al parque en cada intervalo de tiempo, garantizando que el número de visitantes no supere la capacidad establecida.

En este contexto, el proyecto se desarrolla como parte del curso de Sistemas Operativos, haciendo énfasis en el uso de procesos, hilos POSIX y mecanismos de comunicación entre procesos mediante pipes nominales. El sistema implementado simula la operación diaria del parque, modelando tanto el comportamiento del servidor como el de los clientes que generan solicitudes de grupos familiares.

1.2 Objetivos del proyecto

Objetivo **general**

Diseñar e implementar un prototipo concurrente de sistema de reservas para el parque Berlín, utilizando procesos, hilos y pipes nominales en C bajo Linux, que permita controlar el aforo por hora y gestionar solicitudes de reserva de manera automática.

Objetivos específicos

Implementar un proceso servidor capaz de:

- Recibir solicitudes de reserva desde múltiples agentes.
- Autorizar, reprogramar o negar reservas según la ocupación y el aforo máximo.
- Simular el paso del tiempo mediante un reloj de simulación basado en hilos.
- Generar un reporte final de la jornada con métricas de uso del parque.

Implementar procesos cliente que:

- Se registren dinámicamente ante el servidor mediante un mecanismo de handshake.
- Lean solicitudes de reserva desde archivos CSV y las envíen de forma secuencial.
- Presenten al usuario las respuestas del servidor.

Aplicar correctamente llamadas al sistema POSIX relacionadas con procesos, hilos y comunicación por pipes nominales, asegurando el manejo adecuado de errores y recursos.

1.3 Alcance y limitaciones

Alcance

El sistema desarrollado permite:

- Simular un solo día de operación del parque, dentro de un rango horario configurable.
- Configurar, al inicio del servidor, la hora de inicio y fin de simulación, la duración en segundos de cada hora simulada y el aforo máximo permitido por hora.
- Manejar múltiples agentes concurrentes, cada uno con su propio archivo de solicitudes, comunicándose con el servidor a través de un pipe nominal común y pipes de respuesta individuales.
- Gestionar reservas de bloques de dos horas consecutivas para cada familia, respetando el aforo en cada franja horaria.
- Registrar y reportar métricas como número de solicitudes aceptadas, reprogramadas y negadas, así como las horas con mayor y menor ocupación.

Limitaciones

- El sistema está orientado únicamente a simulación en línea de comandos; no se implementa interfaz gráfica ni almacenamiento persistente de reservas en disco más allá de la ejecución.
- El formato de entrada se limita a archivos, no se contemplan modificaciones interactivas durante la simulación.

2.Descripción del problema

2.1 Descripción general del sistema de reservas

El problema a resolver consiste en administrar reservas de acceso al parque Berlín para grupos familiares, de forma que:

- Cada solicitud corresponde a una familia que desea entrar al parque a una hora específica y permanecer durante dos horas consecutivas.
- El parque tiene un aforo máximo fijo, que no debe superarse en ninguna de las horas del rango de simulación.
- Las solicitudes pueden llegar desde varios agentes de forma concurrente, y el servidor debe decidir si las acepta en la hora pedida, las reprograma a otro bloque de dos horas disponible o las niega.

El sistema se modela como una arquitectura cliente–servidor:

- El Controlador de Reserva mantiene una agenda interna de ocupación por hora, simula el avance del tiempo y aplica las reglas de negocio.
- Los Agentes de Reserva funcionan como fuentes de solicitudes, leyendo líneas de un archivo y enviando cada una al servidor a través de un pipe nominal.
- Al finalizar la simulación, el servidor produce un reporte consolidado con estadísticas sobre el comportamiento del parque durante el día.

2.2 Controlador de Reserva (servidor)

El Controlador de Reserva es un proceso que:

- Se inicia con parámetros de línea de comandos: hora de inicio, hora de fin, segundos por hora simulada, aforo máximo y ruta del pipe nominal por el cual recibe mensajes.
- Inicializa una estructura de agenda donde se almacena:
 - Hora inicial, hora final y hora actual de simulación.
 - Aforo máximo del parque.
 - Ocupación por hora.
 - Familias que entran y salen en cada hora.
 - Personas atendidas por hora y contadores de métricas.

Inicia un hilo de reloj que:

- Espera el número de segundos configurado por cada “hora simulada”.
- Incrementa la hora actual.

- Invoca un callback que actualiza la agenda: imprime qué familias salen y entran en la hora correspondiente y ajusta la ocupación interna.
- En el bucle principal, el servidor:
- Lee mensajes de tipo MensajeSolicitud desde el pipe principal.
- Distingue entre:

Mensajes de registro:

- Registra la ruta del pipe de respuesta del agente.
- Responde con la hora actual de simulación.

Mensajes de solicitud de reserva:

- Llama a la función intentar_reservar_dos_horas() de la agenda, que decide si la reserva se acepta, se reprograma o se niega.
- Actualiza los contadores de métricas según la respuesta.
- Envía al agente una estructura MensajeRespuesta indicando el resultado y la hora asignada.
- Al llegar a la hora final de simulación, el servidor:
- Marca el fin del día.
- Imprime el reporte de ocupación y métricas globales.
- Envía un mensaje especial de tipo OP_FIN_DIA a todos los pipes de los agentes registrados para indicar el cierre del sistema.

2.3 Agentes de Reserva (clientes)

Cada Agente de Reserva es un proceso independiente que representa un “operador” o módulo externo que solicita reservas al parque. Su comportamiento es el siguiente:

Se invoca con parámetros de línea de comandos: nombre del agente, ruta del archivo de solicitudes y ruta del pipe nominal del servidor.

Construye un pipe nominal propio de respuesta, único por agente y por proceso, y lo crea en el sistema de archivos.

Ejecuta una fase de registro:

- Envía un mensaje de tipo OP_REGISTRO al servidor, indicando su nombre y la ruta de su pipe de respuesta.
- Espera la respuesta del servidor con la hora actual de simulación y la muestra en consola.

Una vez registrado, el agente:

- Abre el archivo CSV y recorre cada línea.
- Divide cada línea en: Familia, Hora, Personas.

En caso de que la hora del archivo sea menor que la hora actual de simulación que el agente conoce, emite una advertencia local, pero de todos modos envía la solicitud al servidor.

Envía una solicitud al servidor y espera la respuesta en su pipe de respuesta.

Muestra en consola si la reserva fue:

- Aceptada en la hora solicitada.
- Reprogramada a otra hora.
- Negada por ser extemporánea.
- Negada por falta de cupo o porque no hay ningún bloque disponible.

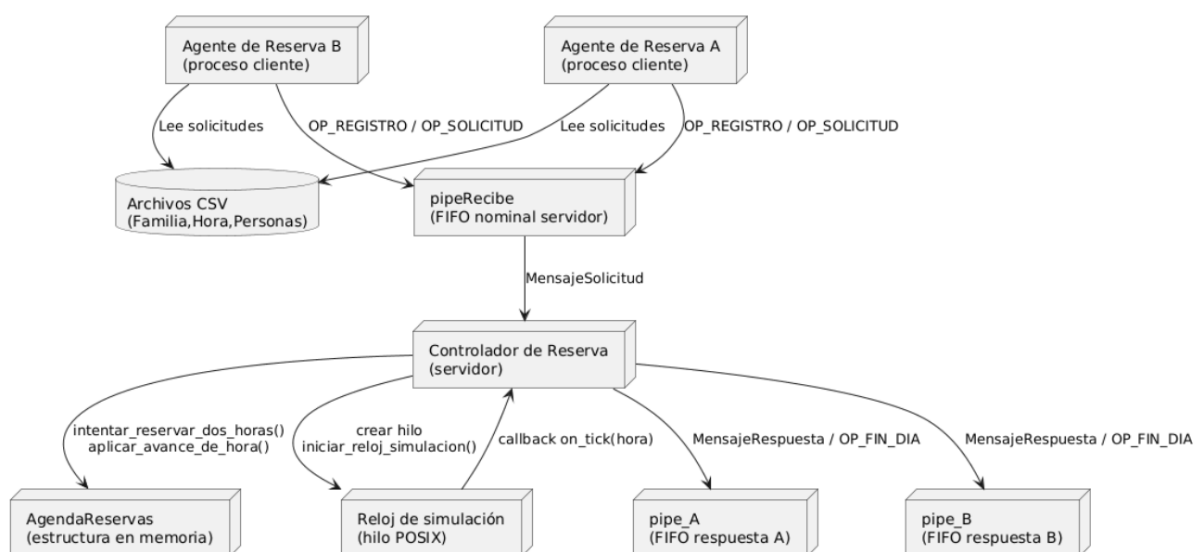
Espera 2 segundos antes de procesar la siguiente línea, simulando un flujo más realista de solicitudes.

Cuando llega al final del archivo:

Informa que el agente ha terminado de procesar su archivo.

Permanece a la espera de un mensaje de tipo OP_FIN_DIA desde el servidor, el cual indica que la simulación del día ha terminado y que el agente puede finalizar su ejecución.

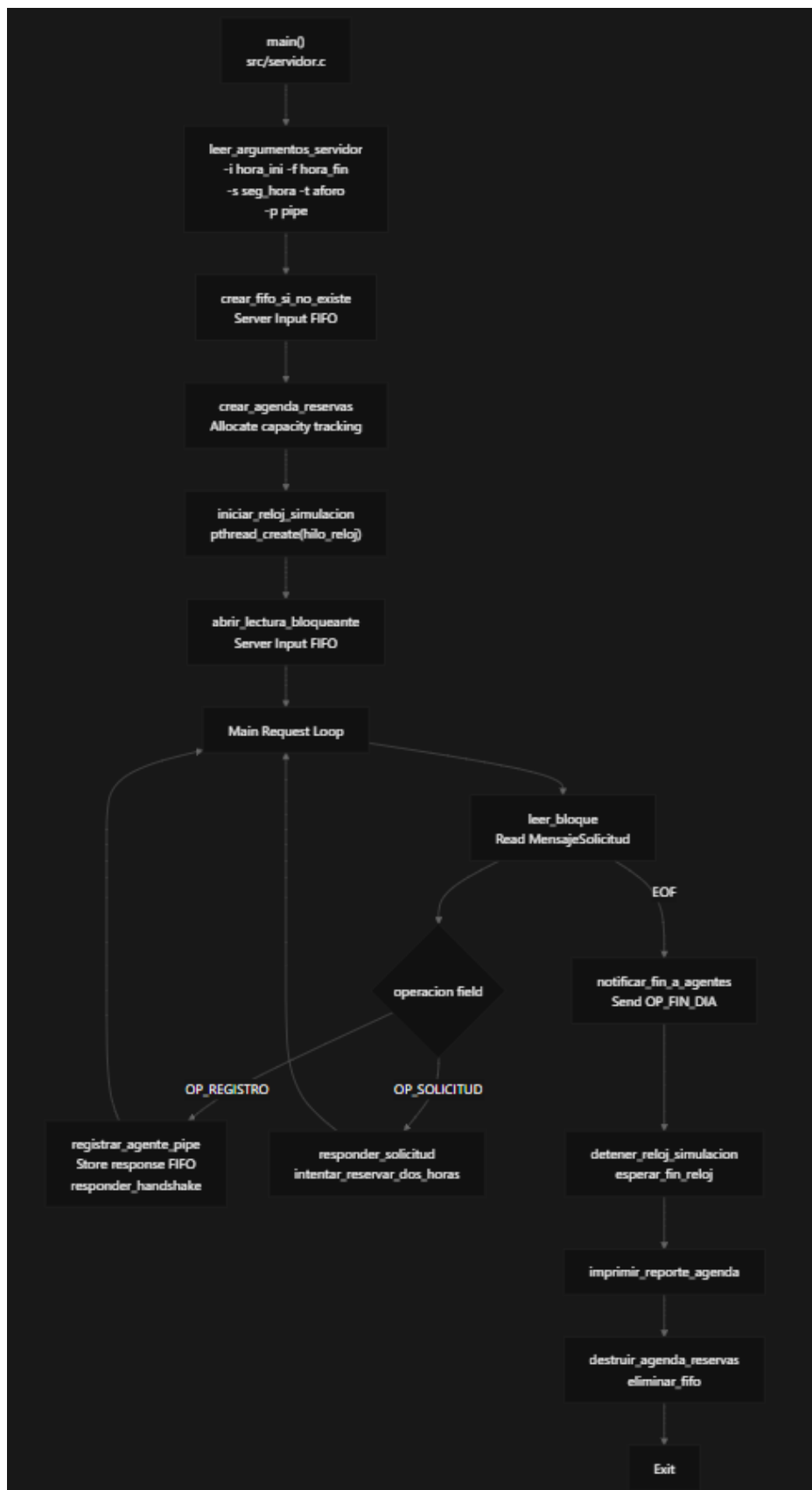
Diagrama de arquitectura de cliente-servidor e hilos:



En el diagrama se presenta la arquitectura general del sistema de reservas, donde se

observa la relación entre el Controlador de Reserva, los Agentes, el reloj de simulación y los pipes.

Modelo de ejecución



Flujo de ejecución del agente



2.4 Reglas de negocio y condiciones del sistema

Las principales reglas de negocio implementadas en el sistema son:

- Duración fija de las reservas:
Cada reserva tiene una duración de exactamente dos horas consecutivas. Si una familia solicita la hora h , su reserva potencial abarca las horas h y $h+1$.
- Rango horario válido:
La simulación se realiza en un rango de horas entero, las reservas solo son válidas si tanto la hora solicitada como la hora siguiente pertenecen al intervalo configurado.
- Control de aforo por hora:
Para aceptar una reserva se verifica que, al sumar las personas de la nueva solicitud en las horas h y $h+1$, el aforo máximo no sea superado en ninguna de las dos.

Clasificación de respuestas:

- Reserva aceptada:
La solicitud se puede ubicar exactamente en la hora solicitada, manteniendo el aforo.
- Reserva reprogramada:
No hay cupo en la hora solicitada, pero sí existe otro bloque de dos horas consecutivas disponible dentro del periodo de simulación. El sistema ubica a la familia en el primer bloque que tenga cupo.
- Reserva negada por extemporánea:
La hora solicitada es menor a la hora actual de simulación. En este caso, el sistema intenta reprogramar dentro del rango restante; si no encuentra bloque disponible, la solicitud se considera negada por extemporánea.
- Reserva negada, debe volver otro día:
La solicitud se niega sin reprogramación cuando:
La hora solicitada está fuera del periodo de simulación.

No existe ningún bloque de dos horas consecutivas disponible para esa familia en el resto del día.

El número de personas en la solicitud excede por sí solo el aforo máximo permitido.

- Simulación del flujo de personas:
El servidor lleva un registro de qué familias entran y salen en cada hora, y ajusta las ocupaciones internas al avanzar el reloj. Al cambiar de hora, se registran en el log las familias que abandonan el parque y las que ingresan en ese instante.
- Métricas de operación:
Al finalizar el día, el sistema calcula:
 - Cantidad de solicitudes aceptadas en la hora solicitada.
 - Cantidad de solicitudes reprogramadas.

- Cantidad total de solicitudes negadas, distinguiendo entre extemporáneas y por falta de cupo.
- Hora con mayor número de personas y hora con menor número de personas.

2.5 Supuestos y restricciones

En el diseño e implementación del sistema se asumen los siguientes supuestos y restricciones:

Entorno de ejecución:

Se supone un sistema operativo Linux compatible con POSIX, que permite la creación de pipes, uso de hilos POSIX y funciones estándar de E/S.

Formato de entrada:

Los archivos de los agentes siguen el formato Familia,Hora,Personas. El sistema implementa validaciones básicas, pero se asume que los valores son coherentes.

Aforo constante:

El aforo máximo por hora se mantiene constante durante toda la simulación del día; no se contemplan cambios dinámicos de capacidad.

Duración fija de las reservas:

Todas las reservas son de dos horas; no se gestionan duraciones variables ni cancelaciones.

Un solo día de simulación:

El sistema está diseñado para simular una única jornada diaria. No se maneja historial ni encadenamiento de días.

Sin tolerancia de fallos compleja:

El servidor maneja condiciones de EOF y reabre el pipe principal cuando es necesario, pero no contempla mecanismos avanzados de recuperación ante fallos de hardware, caídas abruptas del sistema o corrupción de archivos de entrada.

3. Diseño del sistema

Modelo de concurrencia

El servidor utiliza una arquitectura de dos hilos con estado compartido protegido por mutex:

- Hilo principal, src/servidor.c, Procesamiento de solicitudes, registro de agentes, E/S FIFO
- Hilo del reloj, src/reloj_simulacion.c, Avance de tiempo, devoluciones de llamada cada hora

Recurso compartido: AgendaReservas estructura

- Protección: pthread_mutex_t en src/agenda_reservas.c

Operaciones bloqueadas:

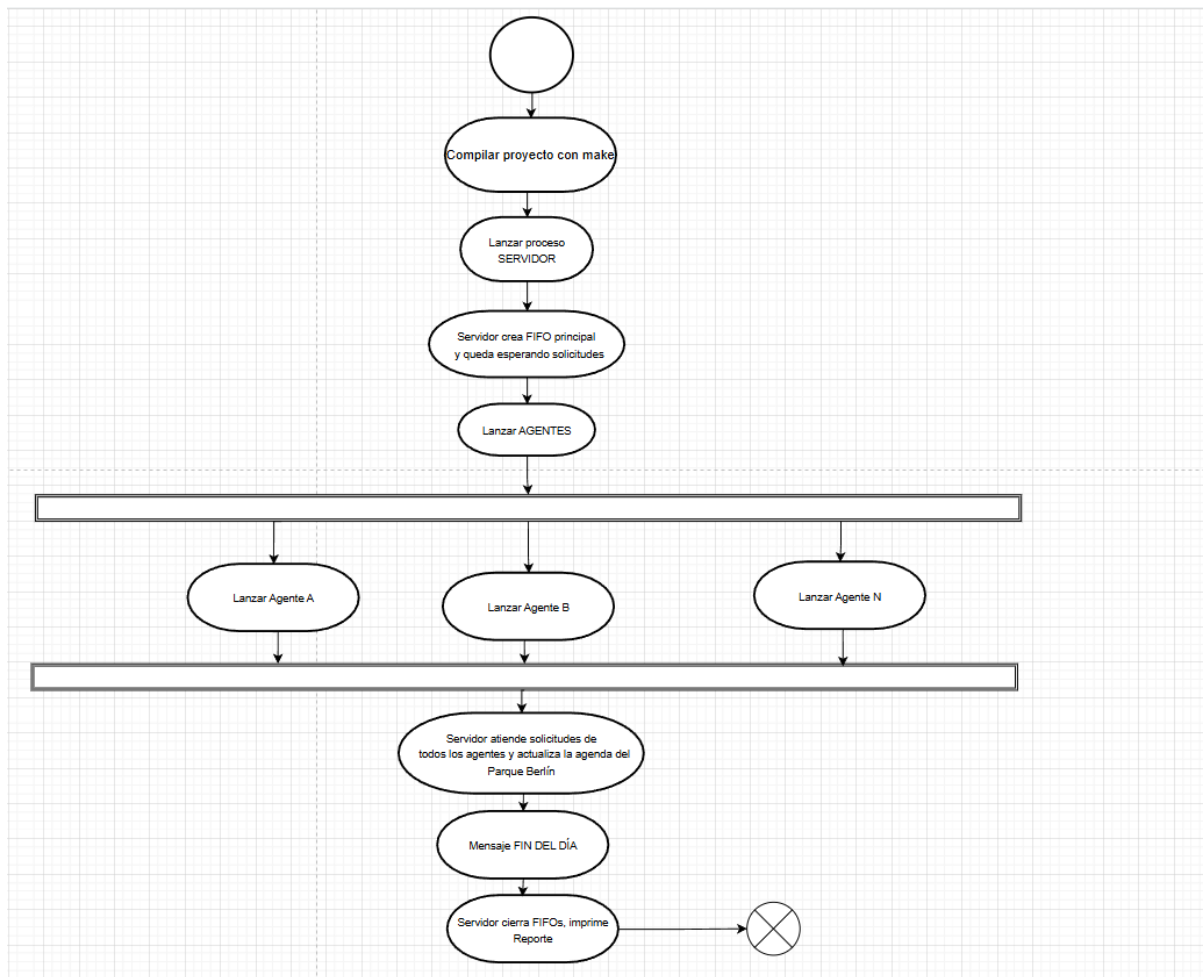
- src/agenda_reservas.c
- src/agenda_reservas.c
- src/agenda_reservas.c

Primitivas de sincronización:

- Bloqueos mutex para acceso a la agenda
- Bandera atómica para la señalización de apagado del reloj
- Bloqueo de E/S FIFO para la coordinación entre procesos
- Los agentes son procesos de un solo hilo que utilizan E/S síncrona con sleep(2) limitación de velocidad entre solicitudes.

3.1 Arquitectura general

El sistema se diseñó siguiendo una arquitectura cliente–servidor clásico, adaptada a un entorno de Sistemas Operativos donde se busca practicar el uso de procesos, comunicación mediante pipes y sincronización con hilos. En este contexto, el servidor representa al “Controlador de Reservas” de una atracción turística: es el encargado de mantener el estado global del día (aforo, horario, ocupación por hora, estadísticas) y de tomar decisiones centralizadas sobre las solicitudes de reserva. Por otro lado, cada agente funciona como una “agencia de viajes” independiente. Cada agente corre en su propio proceso, lee un archivo CSV con las peticiones de sus clientes (familias) y envía estas solicitudes al servidor para que sean evaluadas. De esta manera, se tiene un conjunto potencialmente variable de clientes que generan carga sobre un único componente central que administra un recurso compartido: la atracción con aforo limitado.

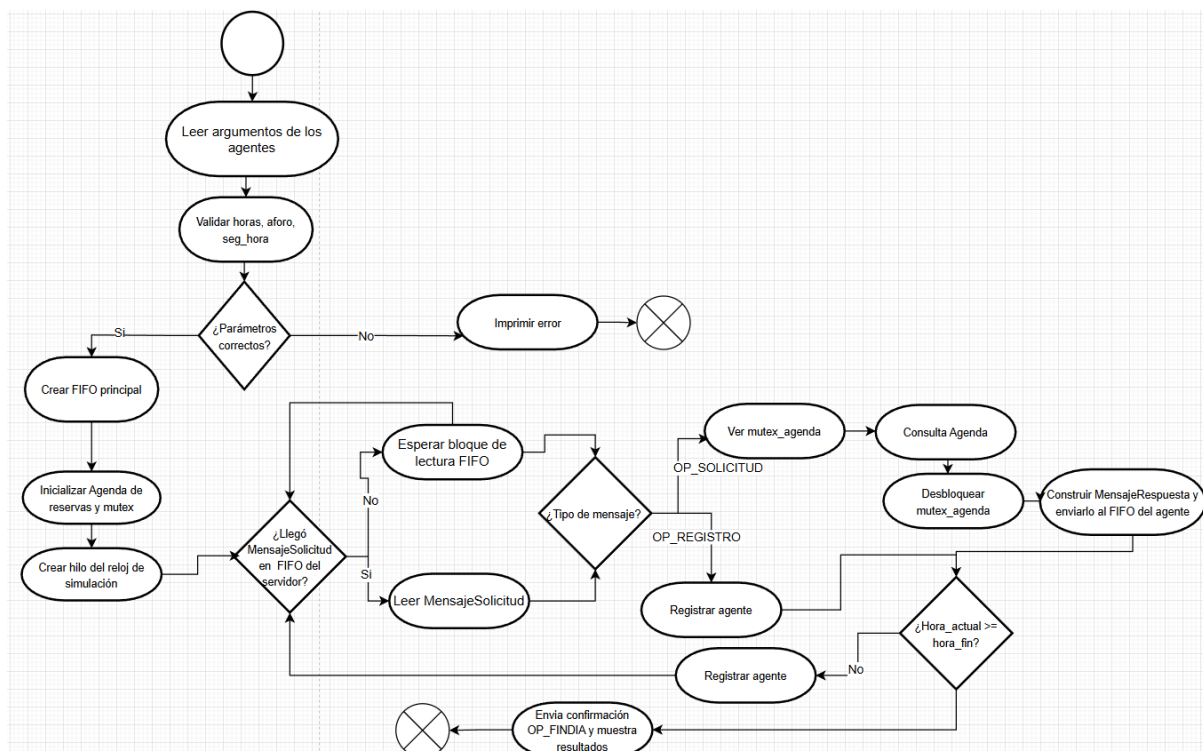


La arquitectura se apoya fuertemente en la comunicación entre procesos mediante pipes nominales (FIFOs). Existe un pipe principal en el que todos los agentes escriben sus mensajes al servidor, y cada agente crea además un pipe exclusivo para recibir las respuestas del servidor. Esto permite separar claramente los flujos de datos: el servidor solo lee solicitudes desde un canal y responde a cada agente usando el canal que ese mismo agente declaró en su registro. Además, dentro del servidor se utiliza un hilo adicional que actúa como reloj de simulación. Este hilo avanza la hora del día de forma controlada (por ejemplo, un segundo real por cada hora simulada) y notifica a la estructura de agenda para que actualice entradas, salidas y métricas. Así, la arquitectura combina procesos independientes (servidor y agentes) con concurrencia interna por hilos (en el servidor), reflejando una situación realista en la que varios clientes interactúan simultáneamente con un servicio centralizado.

3.2 Diseño del servidor

El servidor está diseñado como un proceso que integra tres piezas principales: la agenda de reservas, el reloj de simulación y el bucle principal de atención de mensajes. La

agenda de reservas es una estructura de datos que encapsula todo el estado relevante del día: almacena la hora inicial, la hora final y la hora actual de simulación, guarda el aforo máximo permitido por la atracción y mantiene arreglos que representan cuántas personas hay ocupando cada franja horaria, y conserva listas de “entradas” y “salidas” de familias, de forma que, cuando avanza la hora, se sepa qué familias deben entrar al atractivo y cuáles deben abandonar el lugar. Además, la agenda lleva contadores de cuántas solicitudes fueron aceptadas directamente, cuántas se reprogramaron, cuántas se negaron por extemporáneas y cuántas se negaron por falta de cupo u otras restricciones. Todo este estado queda escondido detrás de funciones como “intentar reservar dos horas”, “aplicar avance de hora” o “imprimir reporte final”, lo cual favorece la modularidad.



El reloj de simulación se implementa como un hilo POSIX que se crea al iniciar el servidor. Este hilo entra en un bucle sencillo: duerme la cantidad de segundos reales configurada para representar una “hora simulada”, incrementa la hora actual y, en cada incremento, invoca una función de callback que se conecta con la agenda. Esa función de callback se encarga de actualizar la hora actual en la agenda, procesar las familias que deben entrar o salir en ese momento y registrar en consola los cambios. Por último, el hilo principal del servidor se encarga de la lógica de comunicación: abre el FIFO de entrada y se bloquea leyendo estructuras de tipo “mensaje de solicitud”. Cada vez que llega un mensaje de registro, el servidor guarda la información del agente (nombre y ruta del pipe de respuesta) y le responde con un mensaje de confirmación donde incluye la hora de simulación actual. Cuando recibe una solicitud de reserva, invoca la función de

la agenda que decide si la reserva se acepta en la hora pedida, se reprograma a una hora distinta o se niega, y genera una respuesta informativa para el agente. Al alcanzar la hora final del día, el servidor sale del bucle de atención, imprime un reporte detallado de lo ocurrido y envía un mensaje especial de “fin de día” a cada agente registrado para indicar que la simulación ha concluido.

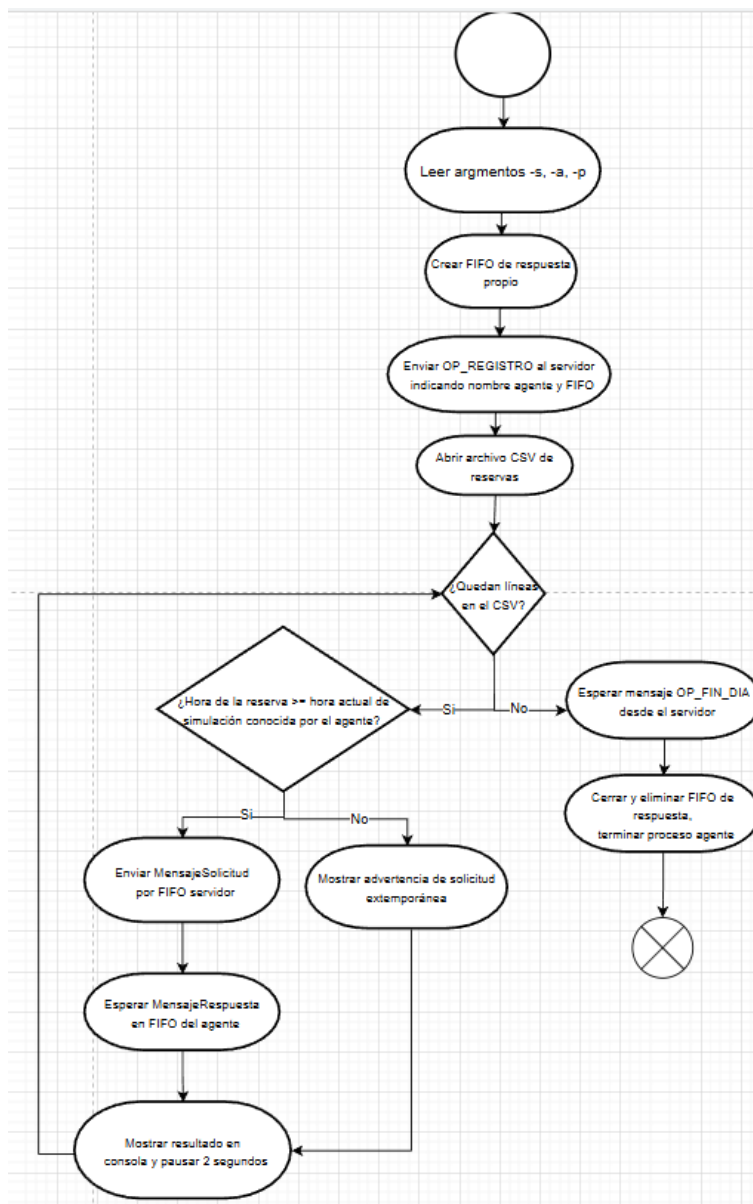
3.3 Diseño de los agentes

Cuando un agente es ejecutado, el primer paso que lleva a cabo es leer y validar los parámetros de línea de comandos: se le indica su correspondiente nombre identificador (-s), la ruta a un archivo CSV con las solicitudes (-a) y la ruta del pipe del servidor (-p). Con toda esa información, el agente está preparado para generar un nombre para su propio pipe de respuesta único (combinando el nombre que tenga el agente y el PID del proceso) y crear este FIFO en el sistema de archivos, un pipe que será usado exclusivamente para que el servidor envíe respuestas a ese agente en particular, evitando así confusiones entre diferentes agentes.

Una vez creado su canal de respuesta, el agente envía un mensaje de registro al servidor por el pipe principal. En ese mensaje indica su nombre y la ruta de su FIFO de respuesta. El servidor le contesta con un mensaje de confirmación en el que se incluye la hora actual de simulación. El agente almacena esta hora como referencia y a partir de allí comienza a procesar el contenido del archivo CSV. Cada línea del archivo describe una solicitud de reserva: el nombre de la familia, la hora que desea visitar la atracción y el número de personas de la familia. Para cada línea, el agente convierte los campos de texto a tipos numéricos, valida que el formato sea correcto y, si detecta que la hora solicitada está por detrás de la hora actual de la simulación, muestra un mensaje de advertencia por pantalla para que el usuario sepa que se trata de una petición extemporánea. Después envía la solicitud al servidor por el FIFO principal y se bloquea esperando una respuesta en su pipe de respuesta. Una vez recibe la respuesta, la imprime de forma legible para el usuario, indicando si la reserva fue aceptada, reprogramada o negada, y en caso de reprogramación, a qué hora quedó finalmente. Entre solicitud y solicitud, el agente introduce una breve pausa (por ejemplo, dos segundos) para simular un flujo de trabajo más realista.

Al terminar de recorrer todo el archivo CSV, el agente informa por consola que ya no tiene más solicitudes que enviar, pero no se cierra de inmediato. En lugar de eso, entra en un bucle donde permanece escuchando en su pipe de respuesta hasta recibir un mensaje especial de tipo “fin de día” proveniente del servidor. Esta decisión de diseño hace que el agente tenga conocimiento explícito de cuándo ha terminado la simulación global, y permite que, en un escenario más realista, el servidor podría enviarle información adicional de cierre. Una vez recibido el mensaje de fin de día, el agente imprime un

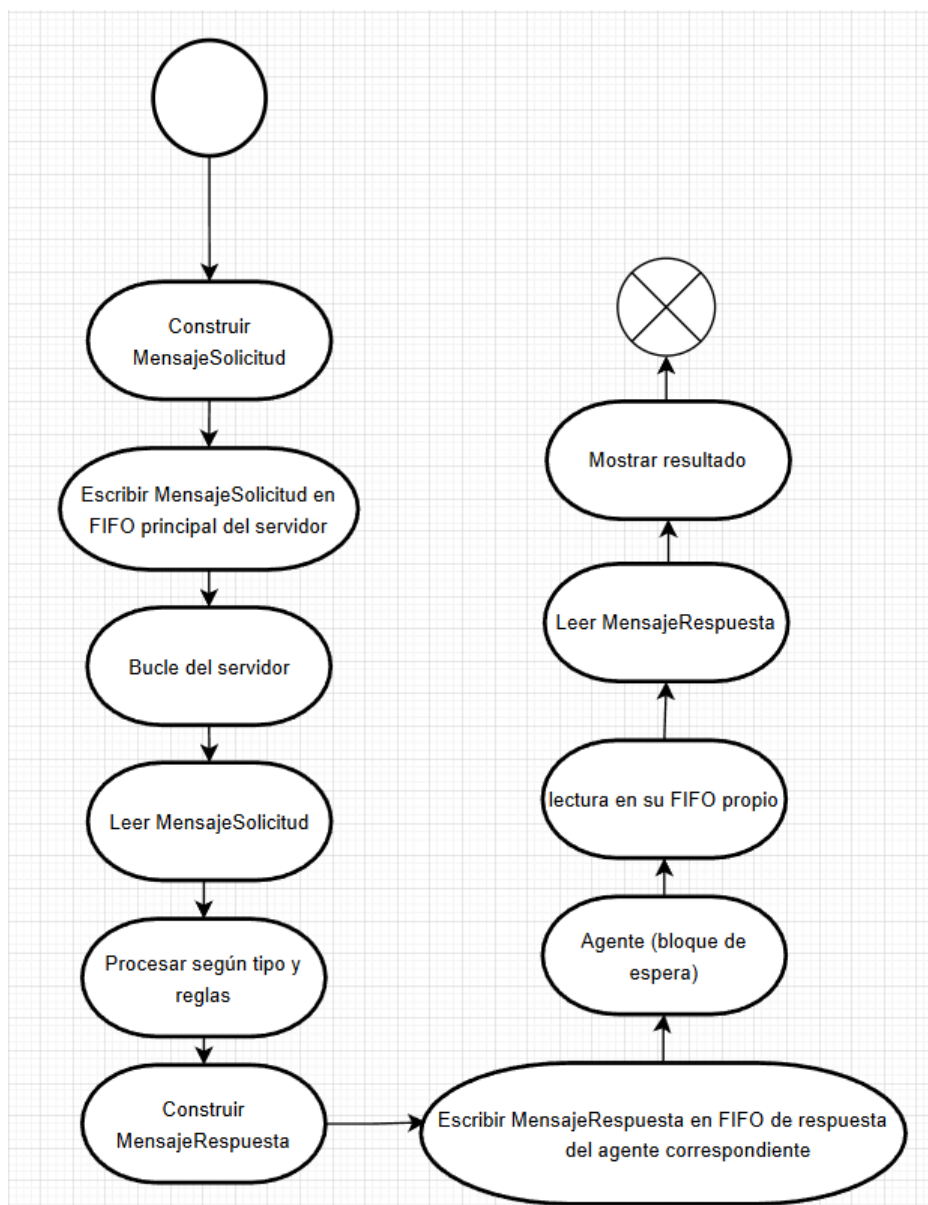
mensaje final, cierra los descriptores de archivo asociados al FIFO y elimina su pipe de respuesta del sistema de archivos, dejando el entorno limpio.



3.4 Comunicación entre procesos

La comunicación entre servidor y agentes se basa completamente en pipes nominales o FIFOs, que son archivos especiales creados en el sistema de archivos mediante la llamada al sistema mkfifo. El servidor, al iniciar, crea el FIFO principal que usará para recibir tanto los mensajes de registro de los agentes como sus solicitudes de reserva. Este pipe se abre en modo bloqueante, de forma que el servidor se queda “esperando” hasta que algún agente escribe un mensaje. Por su parte, cada agente crea su propio FIFO de respuesta en una ruta que no colisione con la de otros agentes, por ejemplo, la ruta puede incluir el nombre del agente y el identificador de proceso, asegurando que cada agente tenga un canal exclusivo.

Para que el intercambio de información sea claro y robusto, se definen estructuras de datos concretas para los mensajes, centralizadas en un archivo de cabecera común. El mensaje de solicitud que los agentes envían al servidor contiene, entre otros campos, el tipo de operación (registro o solicitud), el nombre del agente, la ruta del FIFO de respuesta (en el caso de registro), el nombre de la familia, la hora solicitada y el número de personas. El mensaje de respuesta que el servidor envía a los agentes incluye el tipo de operación (respuesta normal o fin de día), un código que indica el resultado de la operación (aceptada, reprogramada, negada por extemporánea, negada por falta de cupo), la hora asignada si hubo reprogramación y un texto explicativo destinado a mostrar un mensaje claro en la consola del agente.

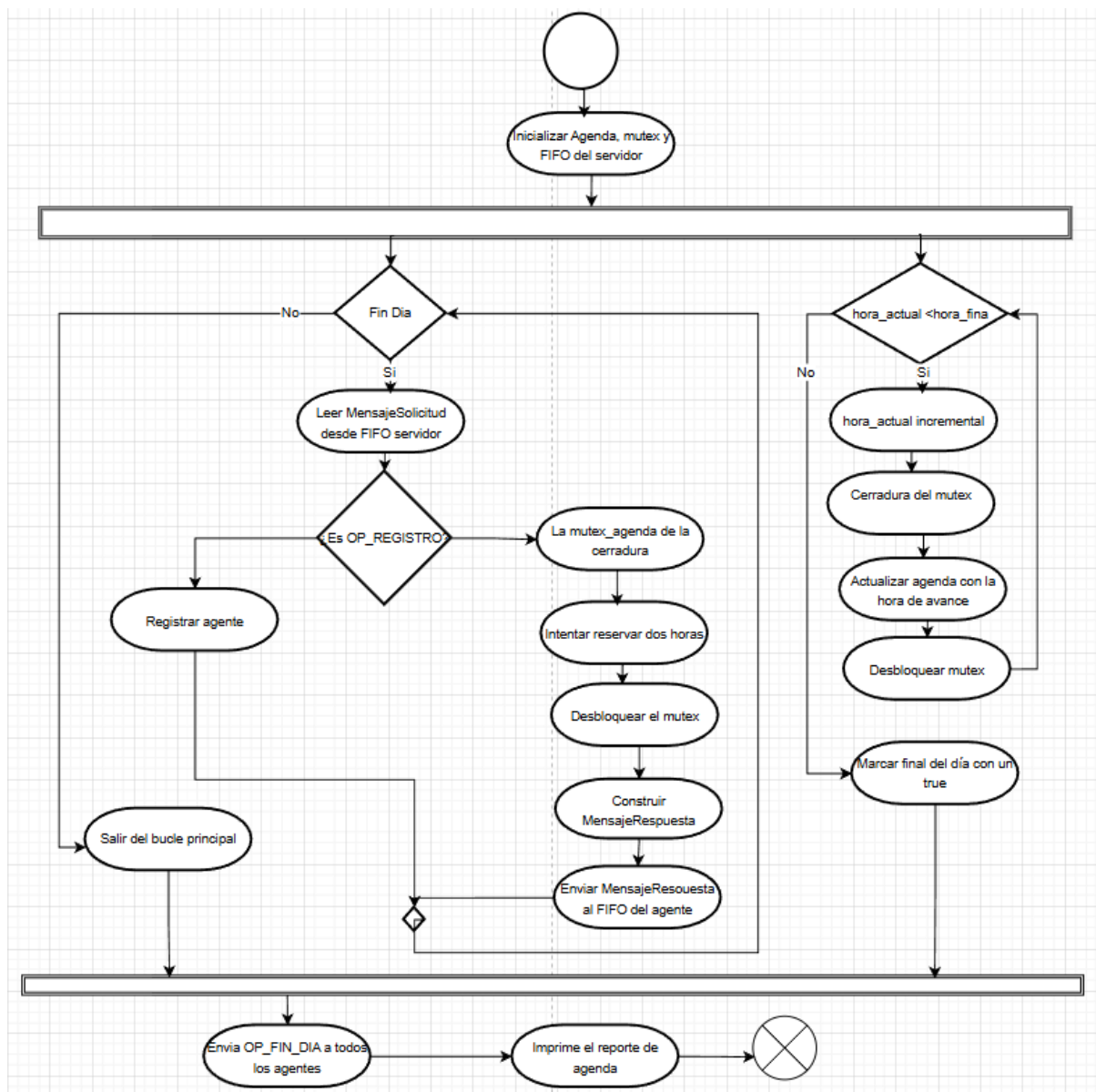


Para enviar y recibir los mensajes de forma segura, el proyecto tiene implementadas funciones auxiliares que se encargan de leer y escribir bloques completos de la memoria sobre los pipes. En vez de depender de una única llamada a read o write, estas funciones

aseguran que se transmitan exactamente el número de bytes de la estructura especificado, reintentando la operación en caso de que la llamada se interrumpa (si ha llegado una señal) o se lean/escriban menos bytes de lo esperado. Así, el servidor y los agentes pueden intercambiar estructuras completas sin el riesgo de que se entremezclen mensajes o se procesen datos incompletos, algo fundamental cuando se utiliza comunicación binaria de bajo nivel.

3.5 Sincronización y concurrencia

El sistema presenta concurrencia en dos niveles diferentes. En primer lugar, en el nivel de concurrencia entre procesos: el servidor y cada agente se ejecutan de manera independiente del sistema operativo y se comunican únicamente a través de los FIFOs. Esto implica que varios agentes pueden leer sus ficheros CSV y enviar solicitudes simultáneamente, los distintos agentes no dependen de la velocidad de ejecución de los otros. El servidor actúa como un recurso compartido: aunque cada agente es un proceso distinto, todos los agentes llegan al mismo FIFO de entrada y todos comparten, a través del servidor, la misma agenda de reservas y el mismo aforo. El servidor procesa las solicitudes una a una en su hilo principal mediante la lógica de negocio y centraliza la agenda de reservas, en lugar de hacerlo desde un proceso de cada agente.



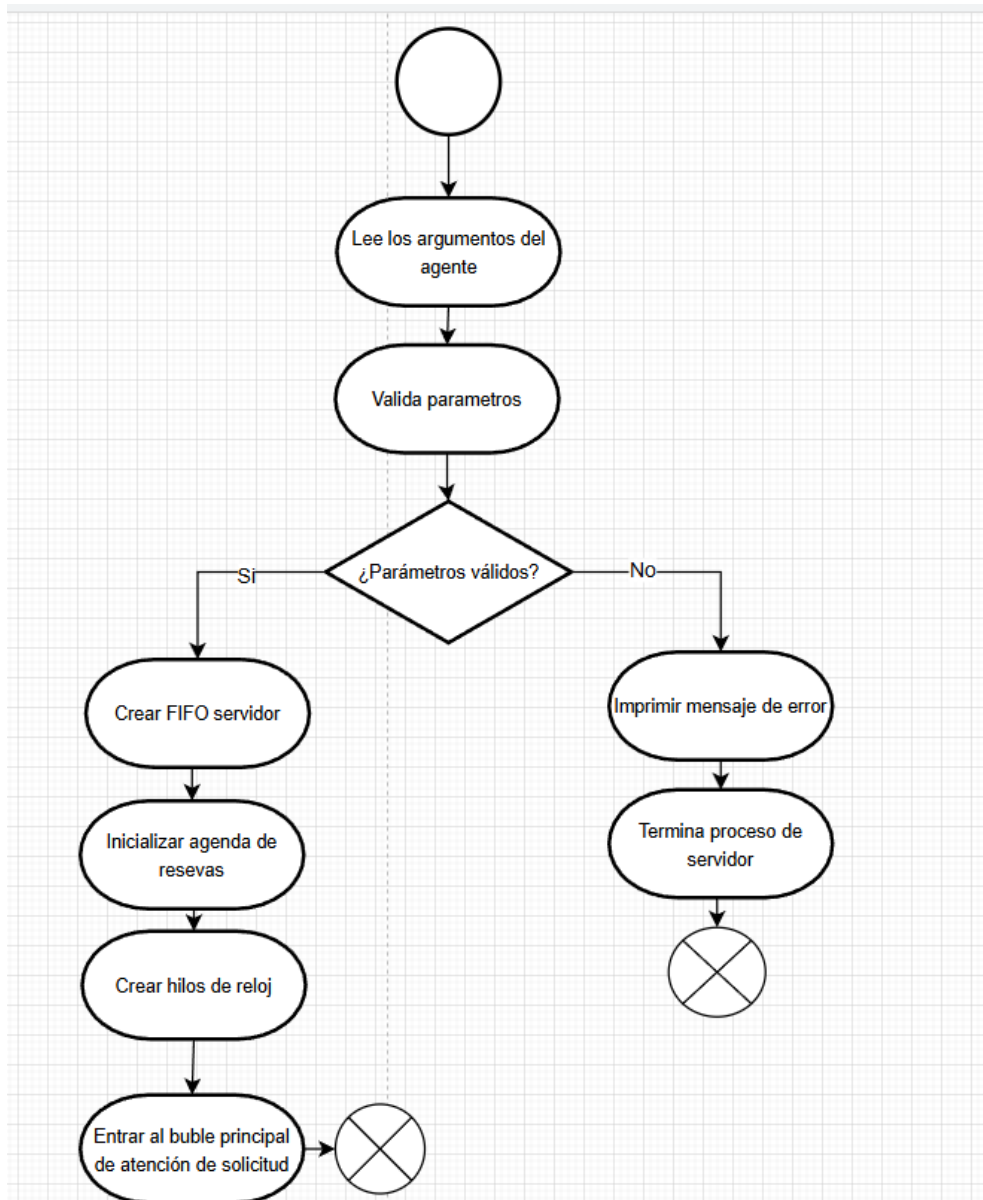
En segundo lugar, existe concurrencia interna dentro del servidor gracias al hilo de reloj de simulación. Este hilo se ejecuta en paralelo con el hilo principal: mientras el hilo principal está bloqueado esperando nuevas solicitudes o procesándolas, el hilo del reloj va avanzando la hora y desencadenando eventos como la entrada y salida de familias. Ambos hilos comparten la misma estructura de agenda, por lo que es necesario coordinar su acceso. Para ello se utiliza un mutex que se asocia a la agenda. Cada vez que se va a modificar el estado de la agenda (al aceptar una reserva, al reprogramarla o al actualizar las entradas y salidas de una nueva hora), se bloquea el mutex antes de acceder a los datos y se libera inmediatamente después. Esto proporciona exclusión mutua, o lo que es lo mismo, que nunca haya dos hilos que estén modificando la misma información a la vez, así se evitan posibles situaciones de competición e inconsistencias. Aparte, se usa una variable de control tipo atómico que sirve de bandera de "fin de día". El hilo del reloj la marca en el momento en que se llega a la hora

final de la simulación, se consulta la variable control para saber el momento en el que hay que salir del bucle principal y realizar las operaciones de cierre y reporte, con lo que se coordina el momento exacto de cuándo debe darse por terminado el sistema.

3.6 Manejo de errores y validaciones

Este proyecto está desarrollado teniendo presente el manejo de errores de forma que el sistema tiene que comportarse de forma predecible incluso cuando las entradas al mismo son incorrectas o en caso de fallo durante las llamadas al sistema. En primer lugar, se lleva a cabo una validación de los parámetros del servidor que consiste en comprobar que la hora inicial y la final de simulación están situadas dentro de un rango aceptado (entre las 7 y las 19 horas), que la hora inicial es estrictamente menor a la final, que el aforo configurado es un número positivo, y que la configuración del número de segundos por hora simulada también sea un valor positivo. De no ser así, el servidor indica el error y no inicia la simulación. De esta forma los agentes validan que se les haya pasado el nombre, el archivo CSV existente y la ruta de pipe del servidor, en caso contrario, muestran un mensaje de uso correcto y terminan.

En la fase de lectura de datos, los agentes también realizan validaciones sobre el contenido del archivo CSV. Cada línea se separa en campos, y se intenta convertir a números la hora solicitada y el número de personas. Si el formato es incorrecto (si la hora no es un entero válido o el número de personas es negativo o cero), el agente puede descartar la línea o advertir al usuario. Además, se revisa si la hora de la reserva ya pasó con respecto a la hora actual de la simulación: en ese caso, el agente lo señala mediante un mensaje de advertencia. Del lado del servidor, la agenda comprueba que las horas solicitadas caigan dentro del rango de simulación y que la cantidad de personas solicitada no supere el aforo, si alguno de estos criterios no se cumple, la reserva se niega o se clasifica como extemporánea, y se registra adecuadamente en las métricas internas.



El sistema también presta atención al manejo de errores de bajo nivel en las llamadas al sistema. Las funciones que crean y abren FIFOs comprueban el valor de retorno de `mkfifo`, si se produce un fallo, utilizan mensajes de error para describir la causa (falta de permisos o inexistencia de la ruta). Las operaciones de lectura y escritura en los pipes utilizan funciones auxiliares que tratan correctamente los casos en los que `read` o `write` se ven interrumpidas por una señal o transfieren menos bytes de los esperados: en lugar de considerar esto como un error silencioso, las funciones reintentan la operación hasta completar el tamaño de la estructura o fallar de forma clara. Finalmente, tanto el servidor como los agentes incluyen rutinas de limpieza de recursos: cierran los descriptores de archivo que ya no se necesitan, destruyen los mutex asociados a estructuras compartidas y eliminan los FIFOs creados al inicio, evitando dejar “basura” en el sistema de archivos y reduciendo el riesgo de fugas de recursos.

4.Implementación

4.1 Lenguaje, compilador y entorno de desarrollo

Se ha llevado a cabo la implementación de todo el sistema en el lenguaje C, de forma que se trabaja de manera directa con las llamadas al sistema propias de Linux, teniendo un control fino de la gestión de memoria, de los descriptores de fichero y de la creación de hilos.

Para la compilación del código se ha usado el compilador **GCC**, configurado con las opciones indicadas en el Makefile del proyecto: `-Wall` para activar la mayoría de *warnings* de compilación, `-pthread` para enlazar correctamente con la librería de hilos POSIX y `-linclude` para que el compilador pueda encontrar las cabeceras del proyecto. Estos *warnings* son útiles para detectar variables no usadas, posibles usos de memoria no inicializada, conversiones de tipo peligrosas, etc., antes de que se conviertan en errores en tiempo de ejecución.

Además, debido a que el servidor utiliza hilos POSIX para el reloj de simulación, el proyecto se compila enlazando contra la librería pthread. El entorno de ejecución es la máquina asignada por la universidad, que utiliza un sistema operativo Ubuntu, ya que se hace uso de llamadas como `mkfifo`, `open`, `read`, `write`, `close`, así como de funciones de la librería pthread. La presencia de un terminal y de herramientas como `gcc` y `make` es suficiente para compilar y ejecutar el proyecto, sin requerir entornos de desarrollo integrados complejos, lo que lo hace adecuado tanto para entornos educativos como para pruebas en máquinas virtuales.

4.2 Estructura de archivos del proyecto

La estructura de directorios del proyecto está ajustada para separar claramente la interfaz de los módulos de su implementación y para organizar los ejecutables, los objetos intermedios y los datos de prueba. Si se ejecuta el comando `tree` en la carpeta raíz del proyecto, se obtiene algo similar a lo siguiente:

```
estudiante@NGEN498:~/proyecto/proyecto$ tree
.
├── bin
│   ├── agente
│   └── servidor
├── build
│   ├── agenda_reservas.o
│   ├── agente.o
│   ├── ipc_fifo.o
│   ├── registro_log.o
│   ├── reloj_simulacion.o
│   ├── servidor.o
│   └── utilidades.o
├── data
│   ├── agenteA.csv
│   └── agenteB.csv
├── include
│   ├── agenda_reservas.h
│   ├── comunes.h
│   ├── ipc_fifo.h
│   ├── registro_log.h
│   ├── reloj_simulacion.h
│   └── utilidades.h
├── Makefile
├── prueba_simple.sh
└── src
    ├── agenda_reservas.c
    ├── agente.c
    ├── ipc_fifo.c
    ├── registro_log.c
    ├── reloj_simulacion.c
    ├── servidor.c
    └── utilidades.c
```

El directorio include/ contiene todos los ficheros (.h) que definen las estructuras, constantes y prototipos de funciones públicos de cada módulo. Por ejemplo, comunes.h define los códigos de operación y las estructuras de mensajes que utilizan tanto el servidor como los agentes, mientras que agenda_reservas.h, ipc_fifo.h, reloj_simulacion.h, registro_log.h y utilidades.h describen las funciones exportadas por cada uno de esos componentes.

Los ficheros que incluyen el código fuente (.c) con la implementación de la lógica que corresponde a cada uno de los módulos del sistema de reservas se encuentran situados en el directorio src/, donde se encuentran agenda_reservas.c, ipc_fifo.c, registro_log.c, reloj_simulacion.c y utilidades.c, además de servidor.c, donde se implementa el proceso servidor, y agente.c, donde se implementa el proceso agente.

El directorio `build/` contiene los archivos objeto (`.o`) generados durante la compilación, de forma que el código fuente permanece separado de los artefactos intermedios. El directorio `bin/` contiene los ejecutables compilados, que son el agente y el servidor, y la carpeta `data/` contiene los ficheros de entrada de prueba (`agenteA.csv`, `agenteB.csv`). Finalmente, el fichero `Makefile`, ubicado en la raíz del proyecto, se encarga de automatizar la compilación y otras tareas, y el script `prueba_simple.sh` facilita la ejecución de pruebas básicas sobre el sistema.

4.3 Instrucciones de compilación

La compilación del proyecto se organiza mediante el fichero `Makefile` situado en el directorio raíz. En dicho fichero se especifica que el compilador a utilizar es `gcc` y se definen unas banderas de compilación comunes (`-Wall -pthread -linclude`) que se aplican a todos los módulos del sistema. A partir de ahí, el `Makefile` declara dos grupos de archivos objeto: por un lado, los que forman parte del servidor (como `build/servidor.o`, `build/agenda_reservas.o`, `build/reloj_simulacion.o`, `build/ipc_fifo.o`, `build/registro_log.o` y `build/utilidades.o`) y, por otro, los que forman parte del agente (principalmente `build/agente.o`, `build/ipc_fifo.o` y `build/utilidades.o`). Estos objetos se generan a partir de los ficheros fuente del directorio `src/`, y quedan almacenados en el directorio `build/`, de forma que el código fuente y los artefactos intermedios permanecen separados.

El objetivo principal del `Makefile` es `all`, que se encarga de construir los dos ejecutables del sistema: `bin/servidor` y `bin/agente`. Para ello, el `Makefile` indica que, cuando se pida construir `bin/servidor`, se deben enlazar todos los archivos objeto del grupo del servidor utilizando el compilador y las banderas definidas; de forma análoga, cuando se solicite construir `bin/agente`, se enlazan los objetos del grupo del agente. La generación de cada archivo objeto se realiza mediante una regla genérica que toma cualquier fichero `*.c` situado en `src/` y lo compila al fichero objeto correspondiente dentro de `build/`, aplicando siempre las mismas opciones de compilación. En la práctica, para compilar el proyecto completo basta con abrir una terminal en la raíz del proyecto y ejecutar el comando `make` (o de forma equivalente, `make all`); esto provoca que se compilen todos los módulos necesarios y que aparezcan los ejecutables ya listos en el directorio `bin/`.

Cuando se desea eliminar los ejecutables y los archivos objeto generados, por ejemplo para forzar una recompilación completa o dejar el árbol de directorios “limpio”, se utiliza el objetivo `make clean`. Este objetivo borra los binarios del directorio `bin/` y los ficheros objeto situados en `build/`, de forma que en la siguiente invocación de `make` sea necesario recompilar todos los módulos desde cero. De este modo, el `Makefile` centraliza y simplifica todas las tareas de construcción del proyecto, evitando que el

usuario tenga que recordar comandos de compilación largos o rutas específicas de cada fichero.

4.4 Instrucciones de ejecución del servidor

Una vez compilado, el servidor se ejecuta desde el directorio raíz del proyecto con una línea de comandos que indica los parámetros de simulación. Un ejemplo de ejecución sería:

```
estudiante@NGEN498:~/proyecto/proyecto$ bin/servidor -i 7 -f 19 -s 1 -t 10 -p /tmp/pipe_reservas
```

En este comando, -i representa la hora inicial en que se da inicio a la simulación, -f es la hora en que se produce el final (en este caso, de 7 a 19), -s indica cuántos segundos reales constituyen una hora simulada (-s 1 significa que, cada segundo que pasa por reloj, avanza una hora en la simulación), -t indica el aforo máximo de personas en el parque (en el ejemplo, 10 personas) y -p da la dirección del FIFO de entrada del servidor, es decir, el pipe donde los agentes envían sus peticiones (/tmp/pipe_reservas).

Cuando el servidor arranca, primero valida que estos parámetros sean coherentes. Después crea la agenda de reservas con la configuración indicada, pone en marcha el hilo del reloj de simulación y crea (si no existe) el FIFO especificado con -p, abriéndolo para lectura en modo bloqueante. A partir de ese momento, el servidor queda esperando mensajes de los agentes y, a medida que estos se registran y envían solicitudes, va generando mensajes informativos en la consola sobre las reservas aceptadas, reprogramadas o negadas, así como sobre las familias que entran y salen de la atracción conforme avanza la hora simulada. Al llegar a la hora final, el servidor imprime un reporte global con estadísticas de todo el día y envía mensajes de “fin de día” a los agentes registrados, tras lo cual realiza la limpieza de recursos y termina su ejecución.

4.5 Instrucciones de ejecución de los agentes

Para activar alguno de los agentes, basta con ejecutar los binarios que se encuentran en el directorio bin/, indicando el nombre del agente, el archivo de reservas y el pipe del servidor. Un ejemplo de ejecución de un agente sería:

```
estudiante@NGEN498:~/proyecto/proyecto$ bin/agente -s agenteA -a data/agenteA.csv -p /tmp/pipe_reservas
```

En este comando, -s asigna un nombre lógico al agente (en este caso, agenteA), -a indica la ruta al archivo CSV que se quiere que gestione este agente (data/agenteA.csv) y -p indica la ruta del pipe del servidor que se está ejecutando y que coincide con el pipe que se utilizó para ejecutar el mismo servidor (/tmp/pipe_reservas). Durante la ejecución, el

agente comprueba que el fichero existe y que es accesible, crea su FIFO de respuesta en /tmp, se registra ante el servidor y empieza a leer el fichero CSV línea tras línea. En función de cada una de las reservas, se lanza una solicitud y se espera una respuesta, mientras se representa en pantalla el resultado de forma clara, junto con pequeños mensajes de estado que sirven de guía para seguir el flujo de la simulación.

Es importante destacar que se pueden lanzar varios agentes en paralelo, cada uno con su propio archivo de reservas y/o su propio nombre. Por ejemplo, en una segunda terminal se podría ejecutar:

```
estudiante@NGEN498:~/proyecto/proyecto$ bin/agente -s agenteB -a data/agenteB.csv -p /tmp/pipe_reservas
```

De este modo, ambos agentes enviarían solicitudes al mismo servidor, compartiendo el aforo y el horario definidos en él, lo que crea una situación más realista donde varias agencias compiten por un mismo recurso limitado. Al finalizar sus ficheros de entrada, los agentes no se cierran de inmediato: permanecen a la espera de recibir el mensaje de fin de día del servidor; cuando esto sucede, muestran un mensaje final, eliminan sus FIFOs de respuesta y terminan su ejecución de manera ordenada.

5. Plan de pruebas

Para el proyecto se desarrollo un plan de pruebas exhaustivo en el que se prueba el funcionamiento completo e íntegro del proyecto. Teniendo en cuenta los siguientes ítems:

- a. Compilación
- b. Validación de argumentos
- c. Handshake agente-servidor
- d. Múltiples agentes concurrentes
- e. Reservas aceptadas/reprogramadas/negadas
- f. Reloj de simulación
- g. Entradas/salidas por hora
- h. Fin del día

5.1 Objetivo de las pruebas

El objetivo del plan de pruebas es verificar de manera completa y sistemática el correcto funcionamiento del sistema de reservas desarrollado en el proyecto. Se busca verificar que funcione correctamente en todos sus aspectos, requisitos funcionales y no funcionales, incluyendo:

- La comunicación entre procesos
- La lógica de reservas, donde se tiene que respetar las reglas estipuladas en el enunciado del proyecto.
- El correcto funcionamiento del reloj de simulación
- La robustez del sistema ante errores de parámetros o de funcionamiento interno.
- Gestión de múltiples agentes concurrentes
- Correcta generación del reporte al final del día.

5.2 Entorno de pruebas

El plan de pruebas está hecho para ejecutarse en sistemas operativos tipo UNIX haciendo uso de los estándares POSIX.

En el caso de este plan de pruebas fue ejecutado en un sistema Ubuntu con arquitectura x86_64 utilizando el compilador GCC (GNU Compiler Collection). Pero de igual manera puede ser ejecutado en MacOS ya sea x86 o ARM, utilizando bien sea GCC o Clang. Ya que las diferencias entre estos 2 sistemas, hablando específicamente para la ejecución de este proyecto, son mínimas.

5.3 Diseño de casos de prueba

Los casos de prueba fueron diseñados siguiendo una estrategia de cobertura incremental y por capas, que va desde lo más básico hasta escenarios complejos de integración.

Empezando por pruebas básicas de compilación y ejecución, pruebas de comunicación entre procesos (IPC), pruebas de la lógica de negocio, pruebas del reloj de simulación, pruebas de finalización.

5.4 Resultados de las pruebas

Teniendo en cuenta los ítems de prueba del punto 5.1 a continuación se muestra la ejecución de diversos casos de prueba, primero se muestra el resultado esperado teórico y después el resultado del programa.

5.4.1 Compilación

Se hace la prueba de compilación de los ejecutables del proyecto haciendo uso del Makefile.

```
```bash
Objetivo: Verificar que el proyecto compila sin errores
make clean
make all

Resultado Esperado:
- bin/servidor creado
- bin/agente creado
- Sin errores de compilación
```
```

Y se comprueba que compila de manera correcta y cumple con las condiciones del resultado esperado.

Antes de compilar:

```
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ tree
.
├── bin
├── build
└── data
```

```
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ make
gcc -Wall -pthread -Iinclude -c src/servidor.c -o build/servidor.o
gcc -Wall -pthread -Iinclude -c src/agenda_reservas.c -o build/agenda_reservas.o
gcc -Wall -pthread -Iinclude -c src/reloj_simulacion.c -o build/reloj_simulacion.o
gcc -Wall -pthread -Iinclude -c src/ipc_fifo.c -o build/ipc_fifo.o
gcc -Wall -pthread -Iinclude -c src/registro_log.c -o build/registro_log.o
gcc -Wall -pthread -Iinclude -c src/utilidades.c -o build/utilidades.o
gcc -Wall -pthread -Iinclude build/servidor.o build/agenda_reservas.o build/reloj_simulacion.o build/ipc_fifo.o build/registro_log.o build/utilidades.o -o bin/servidor
gcc -Wall -pthread -Iinclude -c src/agente.c -o build/agente.o
gcc -Wall -pthread -Iinclude build/agente.o build/ipc_fifo.o build/utilidades.o -o bin/agente
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$
```

Después de compilar:

```
.
├── bin
│   ├── agente
│   └── servidor
├── build
│   ├── agenda_reservas.o
│   ├── agente.o
│   ├── ipc_fifo.o
│   ├── registro_log.o
│   ├── reloj_simulacion.o
│   ├── servidor.o
│   └── utilidades.o
└── data
```

5.4.2 Servidor - Validación de argumentos

5.4.2.1 Caso 1

```
# Caso 1: Argumentos incompletos
bin/servidor
# Esperado: Mensaje de uso y salida con código 1
```

Resultado de la prueba:

```
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ bin/servidor
Uso: bin/servidor -i <horaIni> -f <horaFin> -s <segHoras> -t <aforo> -p <pipeRecibe>
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$
```

5.4.2.2 Caso 2

```
# Caso 2: Hora inicial inválida
bin/servidor -i 6 -f 19 -s 5 -t 50 -p /tmp/pipeReservas
# Esperado: Error de validación (hora_ini < HORA_MIN)
```

Resultado de la prueba:

```
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ bin/servidor -i 6 -f 19 -s 5 -t 50 -p /tmp/pipeReservas
Uso: bin/servidor -i <horaIni> -f <horaFin> -s <segHoras> -t <aforo> -p <pipeRecibe>
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$
```

Al detectar un parametro con un valor invalido imprime el mensaje de uso.

5.4.2.3 Caso 3

```
# Caso 3: Hora final < hora inicial
bin/servidor -i 15 -f 10 -s 5 -t 50 -p /tmp/pipeReservas
# Esperado: Error de validación
```

Resultado de la prueba:

```
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ bin/servidor -i 15 -f 10 -s 5 -t 50 -p /tmp/pipeReservas
Uso: bin/servidor -i <horaIni> -f <horaFin> -s <segHoras> -t <aforo> -p <pipeRecibe>
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$
```

Del mismo modo como en la prueba anterior.

5.4.2.4 Caso 4

```
# Caso 4: Aforo negativo/cero
bin/servidor -i 7 -f 19 -s 5 -t 0 -p /tmp/pipeReservas
# Esperado: Error de validación
```

Resultado de la prueba:

```
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ bin/servidor -i 7 -f 19 -s 5 -t 0 -p /tmp/pipeReservas
Uso: bin/servidor -i <horaIni> -f <horaFin> -s <segHoras> -t <aforo> -p <pipeRecibe>
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$
```

5.4.2.5 Caso 5

```
# Caso 5: Ejecución correcta
bin/servidor -i 7 -f 19 -s 5 -t 50 -p /tmp/pipeReservas &
# Esperado: Servidor inicia y crea /tmp/pipeReservas
```


Resultado de la prueba:

```
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ bin/servidor -i 7 -f 19 -s 5 -t 50 -p /tmp/pipeReservas
```

No arroja mensaje de error y se queda quieta la terminal, puesto a que esta ejecutado el proceso de servidor.

5.4.3 Agente – Validacion de argumentos

5.4.3.1 Caso 1

```
# Caso 3.1: Argumentos incompletos
bin/agente
# Esperado: Mensaje de uso y salida con código 1
```

Resultado:

```
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ bin/agente
Uso: bin/agente -s <nombre_agente> -a <archivo.csv> -p <pipe_servidor>
Ejemplo: bin/agente -s AgenteA -a datos/agenteA.csv -p /tmp/pipeReservas
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$
```

5.4.3.2 Caso 2

```
# Caso 3.2: Archivo CSV inexistente
bin/agente -s AgenteTest -a data/noexiste.csv -p /tmp/pipeReservas
# Esperado: Error al abrir archivo CSV
```

Resultado:

```
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ bin/agente -s AgenteTest -a data/noexiste.csv -p /tmp/pipeReservas
[AgenteTest] Iniciando agente...
[AgenteTest] Pipe de respuesta: /tmp/pipe_AgenteTest_4028875
[AgenteTest] Registrandose con el servidor...
```

Al parecer falla al validar la existencia del archivo

5.4.3.3 Caso 3

```
# Caso 3.3: Ejecución correcta (servidor debe estar corriendo)
bin/agente -s AgenteA -a data/agenteA.csv -p /tmp/pipeReservas
# Esperado: Handshake exitoso, procesamiento de solicitudes
```

Resultado:

```
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ bin/agente -s AgenteA -a data/agenteA.csv -p /tmp/pipeReservas
[AgenteA] Iniciando agente...
[AgenteA] Pipe de respuesta: /tmp/pipe_AgenteA_4030075
[AgenteA] Registrandose con el servidor...
```

Ejecución correcta del agente.

5.4.4 Handshake Agente-Servidor

```
# Terminal 1: Servidor con tiempo largo por hora para observar
bin/servidor -i 7 -f 19 -s 30 -t 50 -p /tmp/pipeReservas

# Terminal 2: Agente
bin/agente -s TestAgent -a data/agenteA.csv -p /tmp/pipeReservas

# Verificar:
# - Agente imprime "Registrandose con el servidor..."
# - Agente imprime "Registrado OK. Hora actual del servidor: 7"
# - Servidor registra "Registro agente TestAgent pipe_resp /tmp/pipe_TestAgent_<PID>"
```

Resultado:

Servidor

```
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ bin/servidor -i 7 -f 19 -s 30 -t 50 -p /tmp/pipeReservas
[21:52:02][pid=4032204][tid=134831460923200][INFO] Servidor listo. Esperando agentes y solicitudes...
[21:52:02][pid=4032204][tid=134831460923200][INFO] Registro agente TestAgent pipe_resp /tmp/pipe_TestAgent_4032327
```

Inicia correctamente y registra al agente de prueba, y muestra su pipe correspondiente y process id.

Agente

```
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ bin/agente -s TestAgent -a data/agenteA.csv -p /tmp/pipeReservas
[TestAgent] Iniciando agente...
[TestAgent] Pipe de respuesta: /tmp/pipe_TestAgent_4032327
[TestAgent] Registrandose con el servidor...
[TestAgent] Registrado OK. Hora actual del servidor: 7
[TestAgent] Leyendo archivo: data/agenteA.csv
[TestAgent] Solicitando reserva: Familia=Zuluaga Hora=8 Personas=5
[TestAgent] RESERVA ACEPTADA en hora solicitada. Hora actual servidor: 7
[TestAgent] Esperando 2 segundos...
```

Se registra correctamente con el servidor y comienza a hacer las solicitudes estipuladas en el csv de prueba.

5.4.5 Múltiples agentes concurrentes

```
# Terminal 1: Servidor
bin/servidor -i 7 -f 19 -s 10 -t 50 -p /tmp/pipeReservas

# Terminal 2: Agente A
bin/agente -s AgenteA -a data/agenteA.csv -p /tmp/pipeReservas &

# Terminal 3: Agente B
bin/agente -s AgenteB -a data/agenteB.csv -p /tmp/pipeReservas &

# Verificar:
# - Ambos agentes se registran correctamente
# - Servidor procesa solicitudes de ambos sin conflictos
# - Cada agente recibe sus respuestas por su pipe único
```

Resultado:

```
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ bin/servidor -i 7 -f 19 -s 10 -t 50 -p /tmp/pipeReservas
[22:03:43][pid=4038338][tid=140683078133568][INFO] Servidor listo. Esperando agentes y solicitudes...
[22:03:43][pid=4038338][tid=140683078133568][INFO] Registro agente AgenteA pipe_resp /tmp/pipe_AgenteA_4038414
[22:03:43][pid=4038338][tid=140683073799744][INFO] Tick -> hora actual 7
[22:03:43][pid=4038338][tid=140683073799744][INFO] Hora 7: sin cambios de entrada salida
[22:03:43][pid=4038338][tid=140683078133568][INFO] Solicitud de AgenteA familia Zuluaga hora 8 personas 5
[22:03:45][pid=4038338][tid=140683078133568][INFO] Solicitud de AgenteA familia Dominguez hora 9 personas 3
[22:03:47][pid=4038338][tid=140683078133568][INFO] Solicitud de AgenteA familia Rojas hora 10 personas 8
[22:03:49][pid=4038338][tid=140683078133568][INFO] Solicitud de AgenteA familia Martinez hora 12 personas 4
[22:03:51][pid=4038338][tid=140683078133568][INFO] Solicitud de AgenteA familia Lopez hora 14 personas 6
[22:03:53][pid=4038338][tid=140683073799744][INFO] Tick -> hora actual 8
[22:03:53][pid=4038338][tid=140683073799744][INFO] Hora 8: entran:
[22:03:53][pid=4038338][tid=140683073799744][INFO] - Zuluaga (5)
[22:03:53][pid=4038338][tid=140683078133568][INFO] Solicitud de AgenteA familia Garcia hora 16 personas 7
[22:04:00][pid=4038338][tid=140683078133568][INFO] Registro agente AgenteB pipe_resp /tmp/pipe_AgenteB_4038569
[22:04:00][pid=4038338][tid=140683078133568][INFO] Solicitud de AgenteB familia Fernandez hora 8 personas 10
[22:04:02][pid=4038338][tid=140683078133568][INFO] Solicitud de AgenteB familia Ramirez hora 10 personas 5
[22:04:03][pid=4038338][tid=140683073799744][INFO] Tick -> hora actual 8
```

El servidor recibe solicitudes de ambos agentes y las gestiona de manera correcta.

```
estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ bin/agente -s AgenteA -a data/agenteA.csv -p /tmp/pipeReservas
[AgenteA] Iniciando agente...
[AgenteA] Pipe de respuesta: /tmp/pipe_AgenteA_4038414
[AgenteA] Registrandose con el servidor...
[AgenteA] Registrado OK. Hora actual del servidor: 7
[AgenteA] Leyendo archivo: data/agenteA.csv
[AgenteA] Solicitando reserva: Familia=Zuluaga Hora=8 Personas=5
[AgenteA] RESERVA ACEPTADA en hora solicitada. Hora actual servidor: 7
[AgenteA] Esperando 2 segundos...
[AgenteA] Solicitando reserva: Familia=Dominguez Hora=9 Personas=3
[AgenteA] RESERVA ACEPTADA en hora solicitada. Hora actual servidor: 7
[AgenteA] Esperando 2 segundos...

estudiante@NGEN140:~/Downloads/proyecto(1)/proyecto$ bin/agente -s AgenteB -a data/agenteB.csv -p /tmp/pipeReservas
[AgenteB] Iniciando agente...
[AgenteB] Pipe de respuesta: /tmp/pipe_AgenteB_4038569
[AgenteB] Registrandose con el servidor...
[AgenteB] Registrado OK. Hora actual del servidor: 8
[AgenteB] Leyendo archivo: data/agenteB.csv
[AgenteB] Solicitando reserva: Familia=Fernandez Hora=8 Personas=10
[AgenteB] RESERVA ACEPTADA en hora solicitada. Hora actual servidor: 8
[AgenteB] Esperando 2 segundos...
[AgenteB] Solicitando reserva: Familia=Ramirez Hora=10 Personas=5
[AgenteB] RESERVA ACEPTADA en hora solicitada. Hora actual servidor: 8
```

Los procesos independientes hacen reservas al mismo tiempo al servidor.

5.4.6 Reservas aceptadas/reprogramadas/negadas

Iniciado el servidor

```
bin/servidor -i 7 -f 19 -s 2 -t 20 -p /tmp/pipeReservas &
```

Iniciando agentes

```
bin/agente -s AgenteA -a data/agenteA.csv -p /tmp/pipeReservas &
```

```
bin/agente -s AgenteB -a data/agenteB.csv -p /tmp/pipeReservas &
```

Utilizando los datos de entrada de prueba agenteA y agenteB se muestran los 3 casos, aceptada, reprogramada y negada. Como se muestra en la salida de la ejecución del programa con estos parámetros.

Aceptada

```
[AgenteB] Solicitando reserva: Familia=Fernandez, Hora=8, Personas=10
[22:55:51][pid=535][tid=8756584640][INFO] Solicitud de AgenteB: familia=Fernandez hora=8 personas=10
[AgenteB] ✓ RESERVA ACEPTADA en hora solicitada. Hora actual servidor: 7
[AgenteB] Esperando 2 segundos...
```

Negada

```
[AgenteA] Solicitando reserva: Familia=Rojas, Hora=10, Personas=8
[22:55:54][pid=535][tid=8756584640][INFO] Solicitud de AgenteA: familia=Rojas hora=10 personas=8
[AgenteA] x RESERVA NEGADA (sin cupo o fuera de rango). Debe volver otro día.
[AgenteA] Esperando 2 segundos...
```

Reprogramada (El mensaje es negada pero la razón es porque ya se encuentra una reserva en esa hora y se reprogramada)

```
[AgenteB] Solicitando reserva: Familia=Gonzalez, Hora=11, Personas=8
[22:55:55][pid=535][tid=8756584640][INFO] Solicitud de AgenteB: familia=Gonzalez hora=11 personas=8
[AgenteB] x RESERVA NEGADA (sin cupo o fuera de rango). Debe volver otro día.
[AgenteB] Esperando 2 segundos...
```

Para los puntos 5.4.7 al 5.4.9 se utilizan estos parametros de entrada para servidor y agente:

```
bin/servidor -i 7 -f 12 -s 2 -t 20 -p /tmp/pipeReservas
```

```
bin/agente -s AgenteA -a data/agenteA.csv -p /tmp/pipeReservas
```

5.4.7 Reloj de simulación

En las salidas de ejecución de prueba del programa se muestra en consola el estado del reloj simulado, el cual según el valor del parámetro S va avanzando de a 1 hora como se ve a continuación:

```
[23:20:03][pid=2641][tid=6095056896][INFO] Tick -> hora actual 10
[23:20:03][pid=2641][tid=6095056896][INFO] Hora 10: salen:
[23:20:03][pid=2641][tid=6095056896][INFO] - Zuluaga (5)
[23:20:03][pid=2641][tid=8756584640][INFO] Solicitud de AgenteA familia Rojas hora 10 personas 8
[23:20:05][pid=2641][tid=6095056896][INFO] Tick -> hora actual 11
[23:20:05][pid=2641][tid=6095056896][INFO] Hora 11: salen:
```

5.4.8 Entradas/salidas por hora

Cuando el reloj llega a una hora de reserva se muestra en consola el mensaje de la entrada o salida de reserva en esa hora especifica, como se puede evidenciar en la imagen:

```
[23:15:32][pid=1901][tid=6158004224][INFO] Tick -> hora actual 10
[23:15:32][pid=1901][tid=6158004224][INFO] Hora 10: salen:
[23:15:32][pid=1901][tid=6158004224][INFO] - Zuluaga (5)
[23:15:32][pid=1901][tid=6158004224][INFO] Hora 10: entran:
[23:15:32][pid=1901][tid=6158004224][INFO] - Rojas (8)
[23:15:32][pid=1901][tid=8756584640][INFO] Solicitud de AgenteA familia Martinez hora 12 personas 4
[23:15:34][pid=1901][tid=6158004224][INFO] Tick -> hora actual 11
[23:15:34][pid=1901][tid=6158004224][INFO] Hora 11: salen:
[23:15:34][pid=1901][tid=6158004224][INFO] - Dominguez (3)
[23:15:34][pid=1901][tid=8756584640][INFO] Solicitud de AgenteA familia Lopez hora 14 personas 6
```

5.4.9 Fin del día

El programa muestra al final del día un mensaje informativo (por el lado del servidor) en el que muestra el resumen general de las reservas del día, cuantas fueron aceptadas, reprogramadas, negadas y el número total de estas.

Servidor

```
[23:15:36][pid=1901][tid=8756584640][INF0] Fin del dia. Imprimiendo reporte...
[23:15:36][pid=1901][tid=8756584640][INF0] ===== REPORTE DEL DIA =====
[23:15:36][pid=1901][tid=8756584640][INF0] Aceptadas en hora: 3
[23:15:36][pid=1901][tid=8756584640][INF0] Reprogramadas : 0
[23:15:36][pid=1901][tid=8756584640][INF0] Solicitudes negadas (total): 2
[23:15:36][pid=1901][tid=8756584640][INF0] Negadas extemp. : 0
[23:15:36][pid=1901][tid=8756584640][INF0] Negadas sin cupo: 2
[23:15:36][pid=1901][tid=8756584640][INF0] Hora con MAYOR numero de personas: 10 (personas=11)
[23:15:36][pid=1901][tid=8756584640][INF0] Hora con MENOR numero de personas: 7 (personas=0)
[23:15:36][pid=1901][tid=8756584640][INF0] =====
```

Agente

```
[AgenteA] Servidor notifica FIN DEL DIA: FIN DEL DIA
Agente AgenteA termina.
```

Se muestra la notificación del fin del día

5.5 Análisis de resultados

El programa pasa perfectamente la mayoría de los casos de pruebas planteados, algunos relacionados con validaciones de datos pasados como argumento como el del nombre del archivo csv, no se están teniendo en cuenta puesto que se asume que el usuario ingresa los parámetros correctos en la ejecución, y estar pendiente de la validación de cada argumento resulta un proceso muy dispendioso.

6. Conclusiones

En este proyecto se implementó un sistema de reservas para el parque Berlín usando una arquitectura cliente-servidor. Un proceso funciona como servidor, encargado de centralizar toda la lógica de las reservas, mientras que varios procesos agentes simulan a las familias que solicitan horarios para asistir al parque.

A lo largo del desarrollo, el equipo fue capaz de incorporar los temas de la materia: creación de procesos, hilos POSIX, comunicación de procesos por medio de FIFOs y sincronización con mutex. Fue adecuado el modo en el que se desarrolló la estructura AgendaReservas para representar el aforo por hora, las entradas, las salidas y, finalmente, las métricas. El hilo de reloj hizo que se adelantase el tiempo de la

simulación independientemente del tiempo real y obligó a tratar correctamente el acceso concurrente a la agenda.

El sistema valida parámetros de entrada, controla que las reservas respeten el horario y el aforo que tenemos configurados y genera un reporte final con las métricas del uso del parque durante el día simulado. No se hacen imprescindibles mecanismos avanzados de tolerancia a fallos, ni es necesario tenerlos en cuenta, dado que el comportamiento se ajusta perfectamente al alcance definido: un único día de simulación, reservas de dos horas y el aforo es un dato constante.

En conclusión, el proyecto cumple su objetivo principal: simular de forma razonablemente realista el proceso de reserva y control de aforo del parque Berlín empleando herramientas propias de sistemas operativos, y deja una base clara para posibles mejoras y extensiones futuras.

7. Archivos de entrada de ejemplo

AgenteA.csv

```
data >  agenteA.csv >  data
1   Zuluaga,8,5
2   Dominguez,9,3
3   Rojas,10,8
4   Martinez,12,4
5   Lopez,14,6
6   Garcia,16,7
7
```

AgenteB.csv

```
data >  agenteB.csv >  data
```

```
1 Fernandez,8,10  
2 Ramirez,10,5  
3 Gonzalez,11,8  
4 Perez,13,6  
5 Sanchez,15,4  
6 Torres,17,9  
7
```